YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMICAL METALLURGY
DEPARTMENT OF MATHEMATICS ENGINEERING

APPLIED SQL
PROJECT REPORT

ISTANBUL METRO DENSITY ANALYSIS SYSTEM (IMDAS)

Group Members:

21058004 İrem Çelebi
21058014 Esma Nur Kibar
21058027 Furkan Gültekin
22052083 Nazlıcan Boz
23052030 Mehmet Emin Hatipoğlu

Instructor: Dr. Fettah Kıran

Github Repository:
https://github.com/iremmclb/istanbul-metro-density-analysis

Istanbul 2026

1. INTRODUCTION

1.1. Problem Definition

Every day, metropolitan transit networks create millions of data points, but this information is often stuck in raw, unstructured logs. The main problem is that there isn't a relational infrastructure that can turn these huge logs into useful information about cities. Urban planners can't easily find capacity bottlenecks or figure out how outside factors affect transit density without a structured database. Additionally, this lack of transparency prevents commuters from anticipating congestion, leading to uninformed travel planning and a stressful daily transit experience.

1.2 Significance of the Project

This project is important for the growth of "Smart Cities" because it turns static data into real-time transit information. The system lets you make data-driven decisions, like finding the best train frequencies and suggesting other ways to get around at busy stations. This is possible because it knows where the network is over-capacity. This lowers the city's operating costs and makes the commute much better for the person who is commuting.

1.3 Feasibilty

The project stays within a realistic and doable scope by looking at the rail network in official municipal datasets. The project makes sure that the engineering work is of high quality and can be done in one semester by focusing on the "CSV to Relational DB" transformation. The architecture uses a multi-table relational model to make sure that the data is correct and that the system is easy to manage for fast queries.

2. TECHNICAL ARCHITECTURE & METHODOLOGY

The database is made by turning large, unstructured transit logs (in CSV format) into a structured relational architecture. Datasets were not used "as-is" during this process. Instead, the unstructured raw data was carefully taken out, noise was reduced, and the filtering was focused on the rail network. After the data was cleaned and normalized, it was put into a centralized relational model. This model was made to protect the data's integrity and let SQL run fast analytical queries. This multi-layered approach makes sure that raw transit entries are turned into a clean space that is good for reporting on complex business intelligence.

2.1 Relational Database Design (ERD)

The IMDAS architecture is based on a multi-table relational model that turns big, unstructured transit logs into a fast analytical environment. The main goal of this design is to keep the data safe while allowing fast SQL queries for real-time reporting.

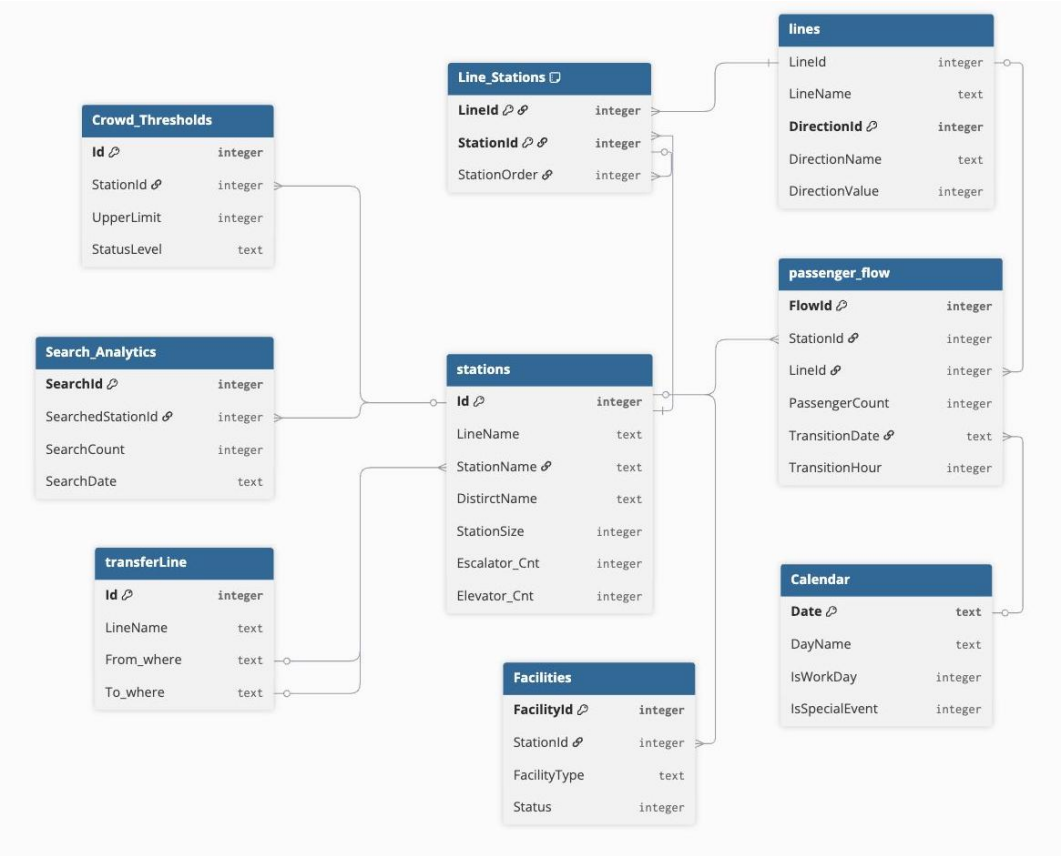

Figure 1 Proposed Relational Database Schema for the IMDAS Project

2.2. SQL Implementation & Analytical Features

The following SQL techniques are planned for implementation:

- **Aggregations:** Using GROUP BY with functions like AVG() and SUM(), we generate periodic reports. This allows us to calculate "Peak Hours" and helping to distinguish between weekday rush hours and weekend travel patterns.

- **JOINs:** We utilize JOINs to unify the high-volume passenger_flow table with stations, lines, and directions tables.

- **CASE WHEN:** Raw passenger counts are meaningful only when interpreted. We implement CASE WHEN statements to dynamically compare real-time counts against the Crowd_Thresholds table.

- **Window Functions :** To provide predictive insights, we use LAG() and LEAD() functions. Additionally, RANK() is used to identify the top 5 busiest stations across the network at any given time.

- **Common Table Expressions**: We use CTEs to break down complex analytical queries into readable, temporary result sets. Specifically, Recursive CTEs are planned for pathfinding logic, allowing the system to traverse the Line_Stations sequence to calculate the total number of stops or transfers between an origin and a destination.

- **Analytical Filtering:** By aggregating data from the Search_Analytics table, we track "Origin-Destination" trends. We specifically filter for short-distance queries (1-3 stops) to identify locations where micro-mobility solutions like scooters or shuttle rings could alleviate metro congestion.

- **Indexing & Views:** We implemented B-Tree Indexing on frequently searched columns like StationID and TransitionDate. Furthermore, we encapsulated complex multi-table joins into Views, providing the application layer with a simplified and high-performance interface for data retrieval.

2.3. Sample Queries

```
SELECT t.LineName, t.From_where, COUNT(t.to_where) transfer_cnt
FROM transferLine t
GROUP BY t.From_where, t.LineName
ORDER BY transfer_cnt DESC
LIMIT 5;
```

This query Identifies the top 5 highest-connectivity transit hubs. Pinpointing these bottlenecks provides actionable intelligence for traffic load balancing and micro-mobility routing.

```
SELECT s.LineName, s.StationName,
SUM(p.PassengerCount) passenger_cnt, S.Elevator_Cnt, s.Escalator_Cnt,
(CASE WHEN SUM(p.PassengerCount) > 150000 AND s.StationSize > 1000
                AND s.Elevator_Cnt < 5
                AND s.Escalator_Cnt < 10 THEN 'Alarm'
        ELSE 'Normal'
 END ) status
FROM  stations s
LEFT JOIN passenger_flow p ON s.LineName = p.LineId
WHERE p.TransitionDate = '2024-08-30'
AND p.TransitionHour BETWEEN 8 AND 20
GROUP BY S.LineName
ORDER BY passenger_cnt DESC
LIMIT 5;
```

This query performs a holiday impact analysis (Aug 30) by correlating passenger volume with physical station infrastructure. It dynamically flags over-capacity stations to enable proactive crowd control.